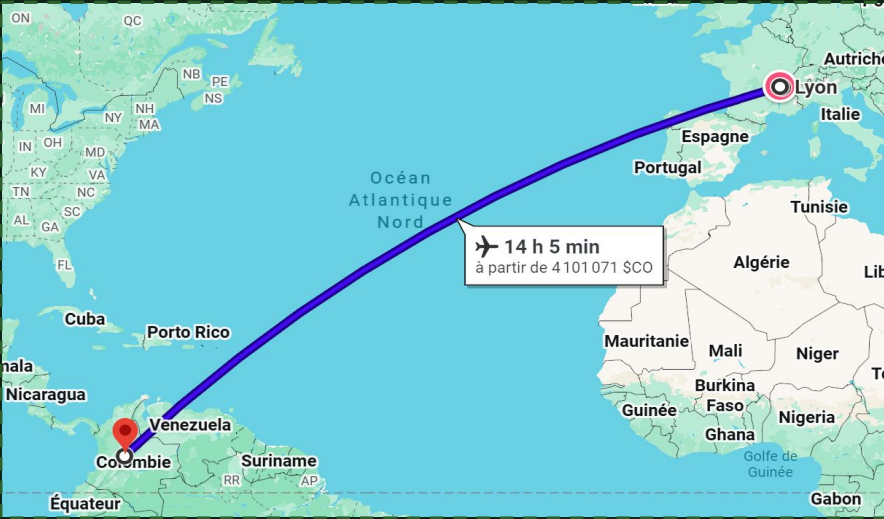
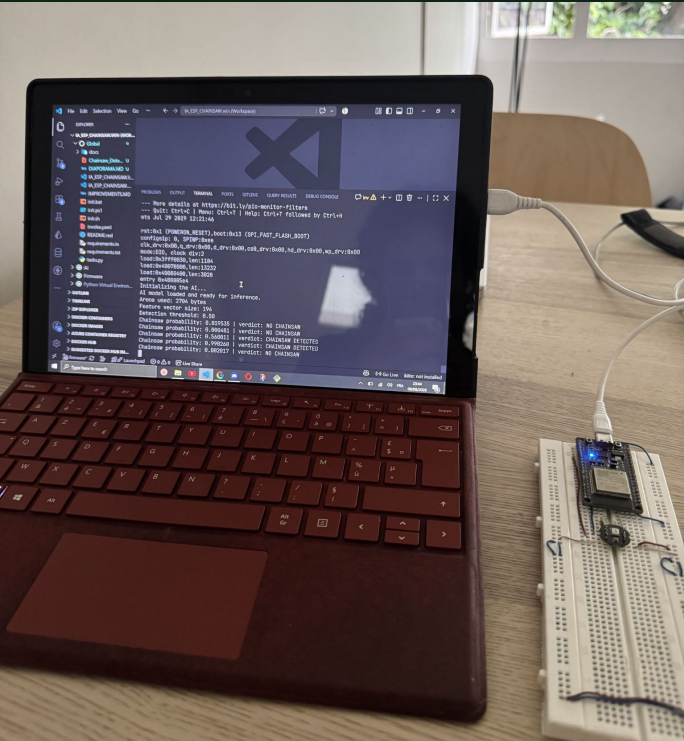


# Hearing the forest

## Acoustic chainsaw detection on an ESP32

*An offline edge-AI node against illegal logging (TensorFlow → TFLite → microcontroller)*

|                |                                                       |
|----------------|-------------------------------------------------------|
| PRESENTED BY   | Rayane Merlin, Engineering student, CPE Lyon (France) |
| APPRENTICESHIP | Energy Pool, Lyon - Energy management                 |
| INTERNSHIP     | Universidad Militar Nueva Granada, Bogotá             |
| PERIOD         | 15 June 2026 → 14 August 2026                         |
| SUPERVISORS    | Edward Guillen · Juan Carlos Martínez                 |



# One command to rule them all

## 1 Bootstrap

*init.sh · init.ps1 · init.bat*

Creates the Python 3.11 venv and installs everything, on Windows, WSL and Linux.

## 2 Locked deps

***pip-tools*** compiles a short, hand-written requirements.in into a fully pinned requirements.txt.

## 3 One task runner

Every action lives in ***tasks.py*** as an Invoke task, each with a short alias.

```
$ inv --list
```

|                                    |                     |                                |                                |                       |                            |
|------------------------------------|---------------------|--------------------------------|--------------------------------|-----------------------|----------------------------|
| <code>inv extract-features</code>  | <code>e, ef</code>  | WAV → feature_dataset.npz      | <code>inv build</code>         | <code>b</code>        | compile with PlatformIO    |
| <code>inv train</code>             | <code>t</code>      | train the Keras model          | <code>inv upload</code>        | <code>u, flash</code> | flash the ESP32            |
| <code>inv convert</code>           | <code>c, tfl</code> | model.h5 → model.tflite        | <code>inv monitor</code>       | <code>m</code>        | live serial detections     |
| <code>inv validate-tflite</code>   | <code>vt</code>     | Keras vs TFLite, per threshold | <code>inv dependencies</code>  | <code>d</code>        | recompile requirements.txt |
| <code>inv generate-firmware</code> | <code>gf</code>     | write the C++ files            | <code>inv full-pipeline</code> | <code>fp</code>       | extract → ... → build      |

From a fresh dataset to a flashed board: `inv full-pipeline` → `inv upload` → `inv monitor`

# From a WAV file to a listening device



## The problem

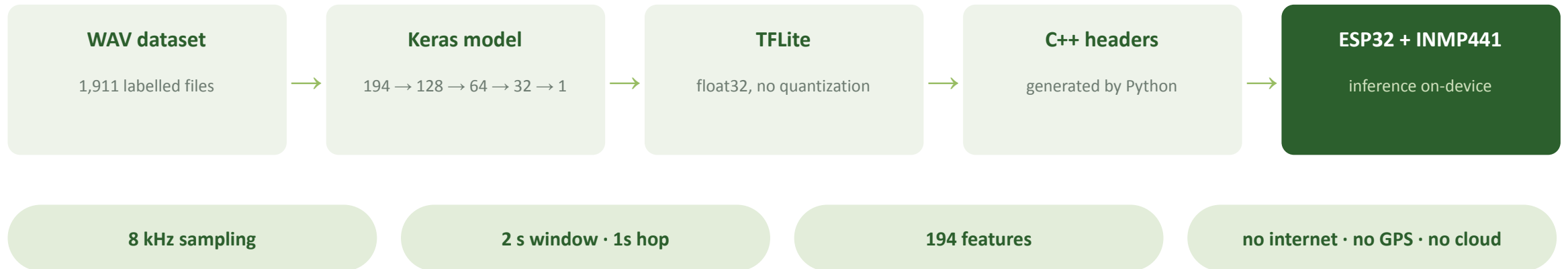
Illegal logging is discovered weeks later, from satellite imagery, when the trees are already gone. In remote forest there is no mobile network, no power grid, no infrastructure.



## The idea

A chainsaw sounds unmistakable: a steady two-stroke engine with a stable harmonic structure, audible far away. Train a network to recognise it, and run that network inside the sensor itself.

### THE CHAIN



*Two fully decoupled modules: training/ (Python, on a PC) · firmware/ (C++, on the board)*

# A five-stage pipeline, fully decoupled



## Re-runnable

Every stage reads the previous stage's output from disk — so any single stage can be re-run alone, without redoing the chain.



## Cache safety

Extraction computes a dataset signature — a hash of every raw file's path, size and mtime — so a stale feature cache is detected automatically.



## Diagnostics

`check.py` verifies cache and label consistency; `chart.py` plots learning curves and the metric history across every training run.

*check.py and chart.py sit alongside the pipeline, they are diagnostics, not stages of it.*

# The dataset: 1,911 recordings and 5,963 windows

1,911

WAV files

5,963

2s windows

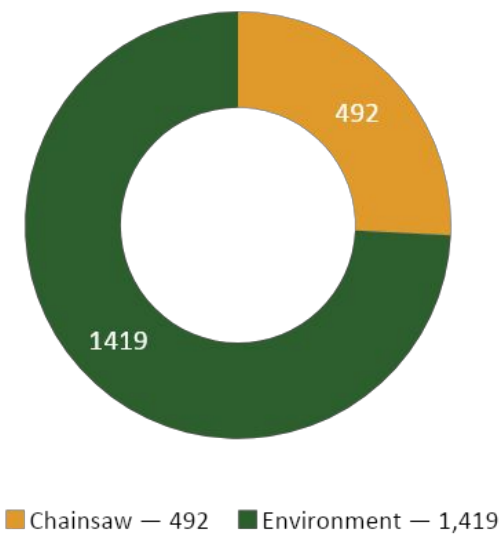
26 %

positive windows

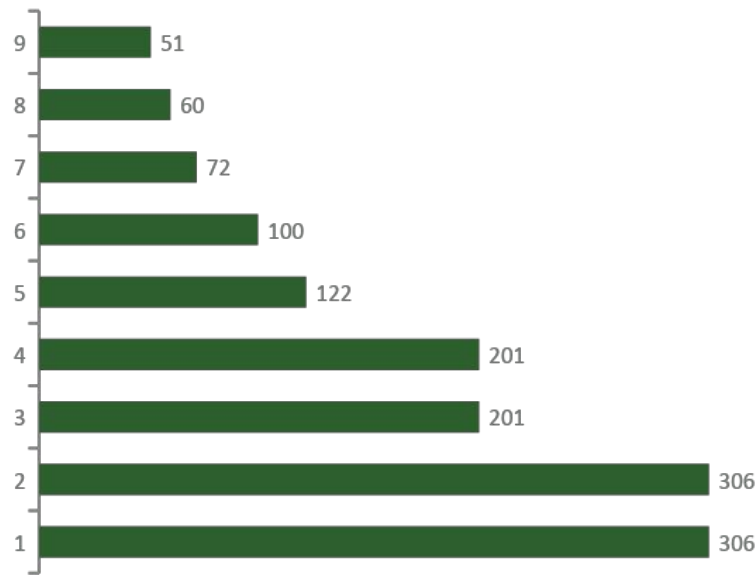
3,580 / 1,197 / 1,186

train / val / test

Dataset composition (files)



Negative categories (files)



### Automatic labelling

Derived from the folder path : chainsaw / motosierra → 1, birds / lluvia / jaguar → 0. A file matching no token raises an error, so the folder tree is the single source of truth.

### Grouped split

All windows from one recording stay on the same side of the split. Otherwise the model would be tested on windows nearly identical to its training data.

### Hard negative

Motocross is deliberately a negative: a two-stroke engine that is not a chainsaw. Class imbalance is handled with weights 0.68 / 1.90.

*Positives come from three saw sources (generic, Oregon, Stihl) plus two sets recorded over jungle ambience.*

# 194 numbers describe two seconds of forest

FEATURE VECTOR : per 2s window



**20 MFCC +  $\Delta$  +  $\Delta^2$**

timbre, and how the timbre evolves



**32-band mel spectrogram (dB)**

coarse spectral envelope



**centroid · bandwidth · rolloff**

where the energy sits in the spectrum



**zero-crossing rate · RMS energy**

noisiness and loudness

*Each series is reduced to (mean, std) over time → a fixed-size vector of 194 values.*

*Single source of truth: the FeatureConfig dataclass — 8 kHz, 2.0 s / 1.0 s, n\_mfcc 20, n\_mels 32, n\_fft 1024, hop 256.*

MODEL : a small dense network

Input (194)

Normalization — adapted on train

Dense 128 · ReLU → Dropout

Dense 64 · ReLU → Dropout

Dense 32 · ReLU

Dense 1 · sigmoid

≈ 35,300

parameters

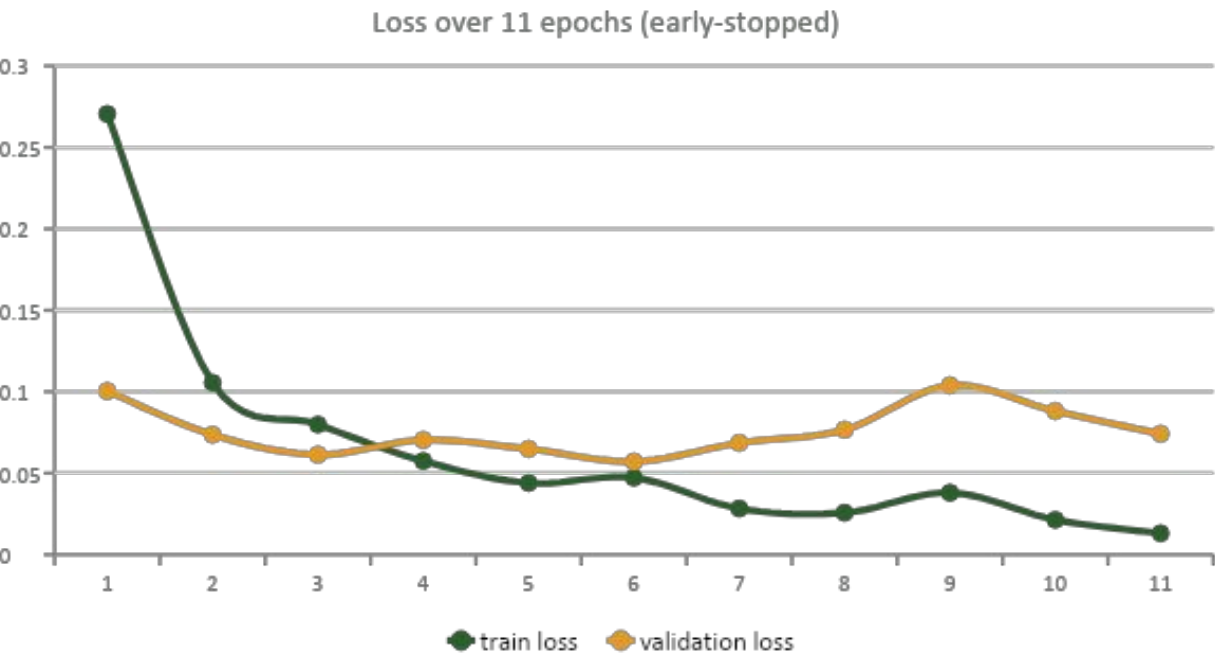
**Adam**

+ binary cross-entropy

**EarlyStopping**

+ ReduceLROnPlateau

# Training results — report #06



**Validation: 98.3 % accuracy · 97.8 % precision · 95.9 % recall · AUC 0.994**

The train / validation gap stays small, no visible overfitting on this dataset. These numbers describe the same kind of audio the model was trained on; they do not yet describe a real forest.

TEST SET 1,186 unseen windows

97.9 %

accuracy

96.1 %

precision

95.8 %

recall

0.995

ROC AUC

Confusion matrix @ threshold 0.5

|     |     |    |    |
|-----|-----|----|----|
| 297 | 864 | 12 | 13 |
| TP  | TN  | FP | FN |

# The conversion is numerically lossless

`validate_tflite.py` runs both models on exactly the same validation and test splits, then compares them window by window.

0

prediction mismatches  
out of 2,383 windows

$5 \times 10^{-9}$

mean absolute difference  
on the output probability

$8 \times 10^{-7}$

worst-case difference  
observed on any window

*That is six orders of magnitude below anything that could change a decision at a 0.5 threshold (pure float32 rounding).*



## The negative result worth reporting

Enabling `Optimize.DEFAULT` applies dynamic-range int8 quantization. It works perfectly with the desktop interpreter and produced **NaN** outputs on TFLite Micro, on the ESP32.

→ the exported model stays plain float32.



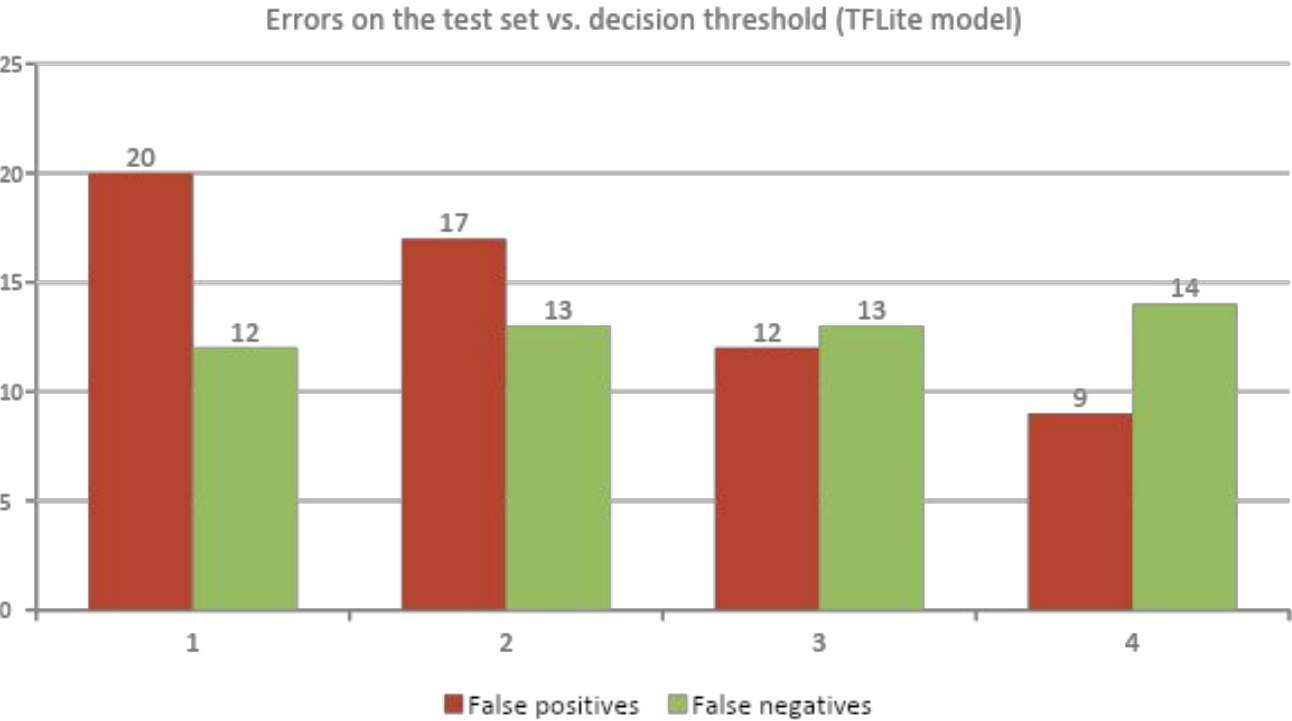
## Report caching

A validation report is only written for a new combination of (reference training report, threshold, SHA-256 of the .tflite file).

→ re-running validation on an unchanged model never produces a redundant, byte-identical report.



# Choosing the threshold: which error do we prefer?



0.30 → 0.60

False positives drop from 20 to 9 : a 55 % reduction for the cost of only two additional false negatives.

THE TWO ERRORS DO NOT COST THE SAME



False negative

A chainsaw we did not hear. A tree is cut.



False positive

A patrol sent into the jungle for nothing and, worse, a device nobody trusts any more.

In the field, alarm fatigue kills a system faster than a missed event  
→ the firmware runs at 0.65

| Thresholds | 0.30   | 0.40   | 0.50   | 0.60   |
|------------|--------|--------|--------|--------|
| Precision  | 93.7 % | 94.6 % | 96.1 % | 97.1 % |
| Recall     | 96.1 % | 95.8 % | 95.8 % | 95.5 % |
| F1 score   | 94.9 % | 95.2 % | 96.0 % | 96.3 % |

# Python writes the C++

*generate\_firmware.py* takes the trained model and the feature configuration, and emits three C++ files.

1

*generate\_feature\_config.py*

*include/app/config.h*

AUDIO\_CONFIG : sample rate, window & hop, n\_mfcc, n\_mels, FFT and hop length.

2

*generate\_dsp\_constants.py*

*src/app/dsp\_constants.cpp*

The mel filterbank and DCT matrices, precomputed as flat float arrays : the same ones librosa builds internally.

3

*generate\_inference.py*

*src/model/inference.cpp*

The .tflite bytes as const uint8\_t *g\_model\_data[]*, with a SHA-256 and size comment for traceability.

## This is the contract between the two modules.

The Python extractor and the C++ extractor cannot drift apart, because both are derived from the same **FeatureConfig** object. And the firmware needs no Python, no TensorFlow and no filesystem — everything is compiled in.

— These three files are generated, never hand-edited and any manual change is silently overwritten. Everything that must survive a regeneration (pins, detection thresholds, watchdog timeout) lives in the hand-maintained *board\_config.h*.

# A layered C++ application on the ESP32

PlatformIO · Arduino framework · C++17 · board nodemcu-32s (ESP32-WROOM-32)

|                                             |                                                                             |
|---------------------------------------------|-----------------------------------------------------------------------------|
| <code>main.cpp</code>                       | orchestrator : setup() and loop()                                           |
| <code>services/</code>                      | mfcc : feature extraction · watchdog                                        |
| <code>drivers/</code> · <code>model/</code> | I2S microphone (INMP441) · generated model bytes + TFLite Micro interpreter |
| <code>app/</code>                           | config.h (generated) · board_config.h (hand-maintained) · flags.h           |

### Only two libraries

The TFLite Micro runtime for ESP32, and a small FFT library. Nothing else.

### Compile-time flags

A disabled feature costs zero bytes of flash and zero CPU cycles, verbose serial output is off by default.

### Fail-safe by design

If a service cannot be initialised, the firmware warns once and keeps running with that feature disabled. On a device meant to sit alone in a forest, never blocking is worth more than being complete.

|                                                                                    |                                                                       |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Generated vs. hand-maintained — the distinction is visible in the file tree itself |                                                                       |
| GENERATED · config.h · dsp_constants.cpp · inference.cpp                           | HAND-MAINTAINED · board_config.h · flags.h · every driver and service |

# Re-implementing librosa in C++, exactly

The firmware must compute the same 194 numbers as the Python extractor (without librosa and without NumPy). Any mismatch and the model receives inputs from a distribution it never saw, and fails silently.

1

## 63 STFT frames

Hann window, reflect padding at the signal edges

2

## Mel energies

MEL\_MATRIX, generated by Python

3

## 20 MFCC

DCT\_MATRIX, generated by Python

4

## $\Delta$ and $\Delta^2$

librosa-equivalent Savitzky–Golay coefficients

5

## Streaming stats

RunningStats — no full-length buffers kept in RAM

!

## The bug that cost me the most time

librosa uses two different decibel references: an absolute one (ref = 1.0) inside mfcc(), and a window-relative one (ref = max) for the standalone mel block.

**I used the same one for both. The model returned a near-zero probability whatever I played at it — the model was fine, the features were not.**

## THE 194-VALUE LAYOUT

```
[ mfcc  $\mu/\sigma \times 20$  ][  $\Delta \times 20$  ][  $\Delta^2 \times 20$  ]
[ mel  $\mu/\sigma \times 32$  ][ centroid ][ bandwidth ]
[ rolloff ][ zcr ][ rms ]
```

*Lesson learned: in embedded machine learning, most bugs are not in the model — they are in feature parity between the training code and the device.*

# The real-time detection loop

SETUP : once

Load the embedded model → build a MicroInterpreter over a 6 KiB tensor arena → AllocateTensors()  
→ init microphone, DSP and watchdog

LOOP : roughly once per second, forever

1

## Sliding window

The first pass records a full 2 s; every pass after that shifts the buffer left by 1 s and only records the newest second.

2

## Features

16,000 int16 samples → 194 floats → copied into the input tensor.

3

## Inference

interpreter->Invoke() → one probability.

4

## Decision

The probability goes through a debounce / hysteresis state machine before anything is raised.

*Inference every second instead of every two → worst-case detection latency halved.*

DETECTION BEHAVIOUR : **board\_config.h**

0.65

**kDetectionEnterThreshold**

probability needed to start counting

2

**kDetectionConsecutiveFrames**

consecutive windows to confirm

0.35

**kDetectionExitThreshold**

probability must fall below this to clear

10 s

**kWatchdogTimeoutSeconds**

the board reboots if the loop hangs

Two thresholds, not one: hysteresis stops the verdict from flickering when the signal hovers at the boundary.

# Excellent on chainsaws, too eager on the rest



## What works

- On real chainsaw sound the model is reliable : recall  $\approx 96\%$ , detection confirmed in about two seconds.
- The whole chain — Keras  $\rightarrow$  TFLite  $\rightarrow$  ESP32 : is validated end to end and runs fully offline.



## What does not

- On sounds that are not chainsaws, the device sometimes fires : false positives are the remaining weakness.
- The model is not wrong: it is being asked about sounds it was never taught to reject.

## MISSING FROM THE DATASET

Wind on the mic

Insects · cicadas

Human voices

Mic self-noise

Thunder

Brush cutters & generators



## And one specific mechanism makes it worse

The firmware peak-normalises every 2 s window. In a near-silent window, the microphone's own noise floor is amplified to full scale : and the model has never been shown that pattern labelled as "not a chainsaw".

# What separates this prototype from a field device

01

## Close the dataset gap

Record negatives on the deployed hardware itself: microphone self-noise, wind, insects, rain on the enclosure, voices, brush cutters. Then multiply the coverage with augmentation, noise mixed at several SNRs, pitch and time shifts, low-pass filtering to simulate distance.

02

## Calibrate the threshold in situ

Today 0.65 comes from an offline study on my own dataset. The right method is to install the node, log its probabilities for several days in the real environment, and pick the operating point that minimises false alarms while keeping recall high. The tooling already exists, every validation run produces a report per threshold.

03

## Then deploy

Target: a node the rangers trust, few enough false alarms that every single alert is acted upon.

*The criterion for success is not accuracy on a test set. It is whether a ranger in the Amazon trusts the alert enough to walk into the forest.*

Thank you, any questions?